

计算机系统概述

本书主要介绍单处理器计算机系统上程序开发和执行的基本原理以及所涉及的重要概念,为高级语言程序员展示高级语言源程序与机器级代码之间的对应关系以及机器级代码在计算机硬件上的执行机制。本章概要介绍计算机的发展历程、计算机系统的基本功能和基本组成、计算机系统层次结构、程序的开发与执行过程以及计算机性能评价的基本概念,此外,还包括对本教材各个章节的组织思路及基本内容的介绍。

1.1 计算机的发展历程

1.1.1 电子计算机的诞生

世界上第一台电子计算机是 1946 年在美国诞生的 ENIAC,其设计师是美国宾夕法尼亚大学的莫齐利 (Mauchly) 和他的学生艾克特 (Eckert)。当时正值第二次世界大战期间,军方急需一种高速电子装置来解决弹道的复杂计算问题,莫齐利与艾克特的方案在 1943 年得到了军方的支持。在冯·诺依曼等人的帮助下,他们经过两年多的努力,终于研制成了第一台电子计算机。1946 年 2 月,美国陆军军械部与摩尔学院共同举行新闻发布会,宣布了第一台电子计算机 ENIAC (Electronic Numerical Integrator and Computer, 电子数字积分机和计算机) 研制成功的消息。

ENIAC 能进行每秒 5000 次加法运算,每秒 50 次乘法运算,以及平方和立方、 $\sin$ 和 $\cos$ 函数数值运算。当时主要用它来进行弹道参数计算,60 秒钟射程的弹道计算时间由原来的 20 分钟一下子缩短到仅需 30 秒,ENIAC 的名声不胫而走。它是个庞然大物,耗资 40 万美元,使用了 18 000 个真空管,重 30 吨,占地面积 170 平方米,耗电 160 千瓦,第一次开机时甚至整个费城地区的照明都闪烁变暗。该机正式运行到 1955 年 10 月 2 日,这十年间共运行了 80 223 小时。

自从第一台电子计算机 ENIAC 诞生后,人类社会进入了一个崭新的电子计算和信息化时代。计算机硬件早期的发展受电子开关器件的影响极大,为此,传统上人们以元器件的更新作为计算机技术进步和划时代的主要标志。

1.1.2 第一代计算机

第一代计算机 (20 世纪 40 年代中到 20 世纪 50 年代末) 为电子管计算机,其逻辑元件采用电子管,存储器件为声延迟线或磁鼓,典型逻辑结构为定点运算。这个时期计算机“软件”一词尚未出现,编制程序所用工具为低级语言。电子管计算机体积大,速度慢 (每秒千次或万

次), 存储器容量小。典型机器除上述的 ENIAC 外, 还有 EDVAC、EDSAC 等。第一台计算机 ENIAC 没有采用二进制操作和存储程序控制, 不具备现代电子计算机的主要特征。

1945 年 3 月, 冯·诺依曼领导的小组发表了“存储程序”(stored-program) 方式的电子数字计算机方案 EDVAC, 宣告了现代计算机结构思想的诞生。“存储程序”方式的基本思想是: 必须将事先编好的程序和原始数据送入主存后才能执行程序, 一旦程序被启动执行, 计算机能不需操作人员干预下自动完成逐条指令取出和执行的任務。冯·诺依曼及其同事在普林斯顿高级研究院 (Institute for Advance Study at Princeton, IAS) 于 1946 年开始设计“存储程序”计算机, 该机被称为 IAS 计算机。

英国剑桥大学的 M. V. Wilkes 在 EDVAC 方案启发下, 于 1949 年制造成功的 EDSAC 成为世界上第一台“存储程序”式的现代计算机, 而 IAS 计算机直到 1951 年才告完成。此外, 尚有 1951 年的 UMVAC-1 和 1956 年的 IBM 704 等也都属于第一代计算机。

1.1.3 第二代计算机

第二代计算机 (20 世纪 50 年代中后期到 20 世纪 60 年代中) 为 晶体管 计算机。1947 年, 美国贝尔实验室的三位科学家 William Shockley、John Bardeen 和 Walter Brattain 发明了晶体管, 为计算机的发展提供了新的技术基础。该实验室于 1954 年研制了晶体管计算机 TRADIC, 而麻省理工学院于 1957 年完成的 TX-2 对晶体管计算机的发展起了重要作用。IBM 公司于 1955 年宣布的全晶体管计算机 7070 和 7090, 开始了第二代计算机蓬勃发展的新时期, 特别是 1959 年 IBM 推出的商用机 IBM 1401, 更以其小巧价廉和面向数据处理的特性而获得广大用户的欢迎, 从而促进了计算机工业的迅速发展。

这一代计算机除了逻辑元件采用晶体管以外, 其内存采用磁芯存储器, 外存采用磁鼓与磁带存储器, 实现了浮点运算, 并在系统结构方面提出了变址、中断、I/O 处理器等新概念。这时计算机软件也得到了发展, 出现了多种高级语言及其编译程序。和第一代电子管计算机相比, 第二代晶体管计算机体积小、速度快、功耗低、可靠性高。

1.1.4 第三代计算机

第三代计算机 (20 世纪 60 年代中到 20 世纪 70 年代中后期) 为 集成电路 计算机。1958 年德州仪器公司的工程师 Jack Kilby 和仙童半导体公司的工程师 Robert Noyce 几乎同时各自独立发明了集成电路, 为现代计算机的发展奠定了革命性的基础, 使得计算机的逻辑元件与存储器均可由集成电路实现。集成电路的应用是微电子与计算机技术相结合的一大突破, 为构建运算速度快、价格低、容量大、可靠性高、体积小、功耗低的各类计算机提供了技术条件。1964 年 IBM 公司宣布世界上第一个采用集成电路的通用计算机 IBM 360 系统研制成功, 该系统的发布是计算机发展史上具有重要意义的事件。该系统采用了一系列计算机新技术, 包括微程序控制、高速缓存、虚拟存储器和流水线技术等; 一次就推出了 6 种机型, 它们相互兼容, 可广泛应用于科学计算、数据处理等领域; 在软件方面首先实现了操作系统, 具有资源调度、人机通信和输入输出控制等功能。IBM 360 系列机的诞生, 对计算机的普及应用和大规模工业化生产产生了重大影响, 到 1966 年年底, 其产量已达到每月 400 台, 5 年内总产量超过 33 000 台。

这一时期还出现了另外一个重要特点，即大型/巨型机与小型机同时发展。1964 年的 CDC 6600 及随后的 CDC 7600 和 CYBER 系列是大型机代表；巨型机有 CDC STAR-100 和 64 个单元并行操作的 ILLIAC IV 阵列机等；小型计算机典型的有 DEC 公司的 PDP 系列。

1.1.5 第四代计算机

第四代计算机（20 世纪 70 年代后期开始）为超大规模集成电路计算机。20 世纪 70 年代初，微电子学飞速发展而产生的大规模集成电路和微处理器给计算机工业注入了新鲜血液。其后，大规模集成电路（LSI）和超大规模集成电路（VLSI）成为计算机的主要器件，其集成度从 20 世纪 70 年代初的几千个晶体管/片（如 Intel 4004 为 2000 个晶体管）到 20 世纪末的千万个晶体管/片。以最大的微处理器制造商 Intel 产品为例（见表 1.1）可以看出，半导体集成电路的集成度越来越高，速度也越来越快，其发展遵循以下摩尔定律：由于硅技术的不断改进，每 18 个月，集成度将翻一番，速度将提高一倍，而其价格将降低一半。戈登·摩尔（Golden Moore）是 Intel 公司的创始人之一，摩尔定律是 1965 年美国《电子》杂志的总编在采访摩尔先生时他对半导体芯片工业发展前景的预测，40 多年来的实践证明，摩尔定律的预测是基本准确的。

表 1.1 Intel CPU 芯片性能比较

芯片名	年代	集成度 (晶体管)	主频范围 (MHz)	线宽 (μm)	数据总线宽 (位)	地址总线宽 (位)
4004	1971	2300	0.74	2	4	
8080	1974	8000	4	1.5	8	16
8086	1978	2.9 万	4.77	1.5	16	20
80286	1984	13.5 万	6 ~ 25	1 ~ 1.5	16	24
80386	1985	27.5 万	20 ~ 33	1 ~ 1.5	32	32
80486	1989	118.5 万	33 ~ 100	1	32	32
Pentium	1993	300 万	60 ~ 133	0.6	64	32
Pentium Pro(P6)	1995	550 万	150 ~ 233	0.6	64	32
PII(P6 + MMX)	1997	750 万	233 ~ 400	0.35	64	32
PIII	1999	950 万	450 ~ 1000	0.25	64	32
P4	2000	4200 万	1500 ~ 3800	0.18	64	32
Core i7	2010	117 000 万	3333	0.032	64	64

随着超大规模集成电路与微处理器技术的长足进步和现代科学技术对提高计算能力的强烈需求，并行处理技术的研究与应用以及众多巨型机的产生也成为这一时期计算机发展的特点。1976 年，Cray 公司推出的 Cray-1 向量巨型机，具有 12 个功能部件，运算速度达每秒 1.6 亿次浮点运算。不少巨型机采用成百上千个高性能处理器组成大规模并行处理系统，其峰值速度已达到每秒几千亿或万亿次，这种并行处理技术成为 20 世纪 90 年代巨型机发展的主流。

第四代计算机时期的另一个重要特点是计算机网络的发展与广泛应用。由于计算机技术与通信技术的高速发展与密切结合，掀起了网络热潮，大量的计算机联入不同规模的网络中，然后通过 Internet 与世界各地的计算机相联，大大扩展和加速了信息的流通，增强了社会的协调与合作能力，使计算机的应用方式也由个人计算方式向网络化方向发展。

由于计算机在技术发展和使用方式上的不断进步，目前学术界和工业界大多已不再沿用传

统以元器件划分“第x代计算机”的提法。

近70年来,人们使用计算机的方式发生了巨大变化。早先是多人共享一台计算机(分时计算方式),然后是一人使用一台计算机(个人计算方式),进而发展到目前多人使用多台计算机的网络计算方式,也就是后PC时代。进一步的发展趋势是人们将进入普适计算(pervasive computing)时代。

计算机技术的另一个重要发展方向是计算机的智能化,其着眼点是发展以知识为基础的智能化处理能力,用以模拟或部分替代人的智能活动,并提供智能化的人机交互接口,使计算机具有自然的人机通信能力。普适化、智能化、嵌入式和网络化将是未来计算机发展的主要方向。

1.2 计算机系统的基本功能和基本组成

计算机是一种能自动对数字化信息进行算术和逻辑运算的高速处理装置。也就是说,计算机处理的对象是数字化信息,处理的手段是算术和逻辑运算,处理的方式是自动的,因此,计算机与算盘以及各类机械式计算器有本质的差别。

通常所说的计算机系统,除了包含看得见的计算机硬件以外,还包括运行在计算机硬件上的软件。

1.2.1 计算机系统的基本功能

计算机系统不仅具有数据处理功能,还具有数据存储、数据传送等功能。

数据处理(data process)是计算机系统最基本的功能。计算机不仅可以进行加、减、乘、除等基本算术运算,也可以进行与、或、非等逻辑运算;处理的数据不仅可以是日常生活中使用的十进制数据,也可以是文字、图形、图像、声音、视频等非数值化的各种多媒体信息。

数据存储(data storage)功能是计算机能采用自动工作方式的基本保证。计算机中提供的存储器使得程序和数据能事先被存储,并在需要时指令被取出自动执行。计算机中有各类存储部件。大量的文件信息需要长期存储在计算机系统中,因此有能够长期保存信息的像磁盘存储器那样的非易失性存储器;正在执行的程序和处理的数据需要存放在快速存储器中,因此有半导体元器件构成的随机访问存储器等。

数据传送(data transfer)功能是指计算机内部的各个功能部件之间、计算机主机与外部设备之间、各个计算机系统之间进行信息交换的操作功能。例如,进行数据处理的部件需要从存储部件中读取数据或写入数据;输入设备的数据需要送到存储部件保存或送到数据处理部件进行计算;一台计算机产生的数据需要送到另一台计算机。因此,计算机系统中不可避免地需要进行数据传送。

数据处理、数据存储和数据传送的功能最终是通过执行指令来完成的,而计算机指令的执行过程由控制器产生的控制信号来控制。

对照上述基本功能,计算机中需要对数据进行处理、存储和传送的基本功能部件以及控制这些功能部件操作的控制部件。通常把进行数据处理的部件称为运算部件或运算器,主要的运算部件是算术逻辑部件(Arithmetic Logical Unit,简称ALU);把进行数据存储的部件称为存储部件或存储器,主要分外存(storage)和内存(memory);把进行数据传送的部件称为互连

部件，主要有总线（bus）、桥接器等。

计算机系统具有高速、通用、准确和智能等特性。计算机的主要核心部件采用高速电子元器件制造，这为计算机快速处理提供了基本保证。通用性体现在两个方面：一是它所处理的信息的多样化，可以是各种数值数据和非数值数据；二是计算机应用极其广泛，只要现实世界中某个问题能找到相应的算法并在有限步骤内完成，就能编制成程序通过计算机执行来加以解决。此外，计算机系统强大的计算和自动逻辑推理能力为计算机系统的准确性和智能化提供了重要基础。

计算机系统所完成的所有任务都是通过执行程序所包含的指令来实现。计算机系统由硬件和软件两部分组成。硬件（hardware）是物理装置的总称，人们看到的各种芯片、板卡、外设、电缆等都是计算机硬件。软件（software）包括运行在硬件上的程序和数据以及相关的文档。程序（program）是指指挥计算机如何操作的一个指令序列，数据（data）是指令操作的对象。

1.2.2 计算机硬件

从 20 世纪 40 年代计算机诞生以来，尽管硬件技术已经经历了四个发展阶段，计算机体系结构也已经取得了很大的发展，但绝大部分计算机的硬件基本组成仍然具有冯·诺依曼结构计算机的特征。冯·诺依曼结构计算机的基本思想主要包括以下几个方面。

① 采用“存储程序”工作方式。

② 计算机由运算器、控制器、存储器、输入设备和输出设备五个基本部件组成。

③ 存储器不仅能存放数据，而且也能存放指令，形式上数据和指令没有区别，但计算机应能区分它们；控制器应能自动执行指令；运算器应能进行加、减、乘、除 4 种基本算术运算，并且也能进行逻辑运算；操作人员可以通过输入/输出设备使用计算机。

④ 计算机内部以二进制形式表示指令和数据；每条指令由操作码和地址码两部分组成，操作码指出操作类型，地址码指出操作数的地址；由一串指令组成程序。

计算机硬件主要包括中央处理器、存储器、外部设备和各类总线等。

中央处理器（Central Processing Unit，简称 CPU）有时简称为处理器，是整个计算机的核心部件，主要用于指令的执行。CPU 主要包含两个基本部分：数据通路和控制器。数据通路（datapath）主要用来执行算术和逻辑运算以及寄存器和存储器的读/写控制等，其中，算术逻辑部件（ALU）用来进行基本的算术和逻辑运算，ALU 中最基本的部件是加法器，所有算术运算都可以基于加法运算和逻辑运算来实现。控制器（controller）用来对指令进行译码，生成相应的控制信号，以控制数据通路进行正确的操作。

存储器分为内存和外存。内存包括主存储器（main memory，简称主存）和高速缓冲存储器（cache）。因为早期计算机中没有 cache，所以一般情况下并不区分内存和主存，两者含义相同，都是特指主存储器。外存包括辅助存储器和海量后备存储器。通常把系统运行时直接和主存交换信息的存储器称为辅助存储器，简称辅存，目前主要的辅助存储器是磁盘存储器和固态硬盘；而磁带存储器和光盘存储器的容量大、速度慢，主要用于信息的备份和脱机存档，因此它们被用作海量后备存储器。

外部设备简称为外设，也称为I/O 设备，其中，I/O 是输入/输出（Input/Output）的缩写。外设通常由机械部分和电子部分组成，并且两部分通常是可以分开的。机械部分是外部设备本

身,而电子部分则是控制外部设备工作的I/O 控制器或I/O 适配器。外设通过 I/O 控制器或 I/O 适配器连接到主机上, I/O 控制器或 I/O 适配器统称为设备控制器。例如, 键盘接口、打印机适配器、显示控制卡 (简称显卡)、网络控制卡 (简称网卡) 等都是一种设备控制器, 属于一种I/O 模块。

小贴士

外部设备种类繁多, 且具有不同的工作特性, 因而它们在工作方式、数据格式和工作速度方面存在很大差异。此外, 由于 CPU、内存等计算机主机部件采用高速元器件实现, 使得它们和外设之间在技术特性上有很大差异, 它们各有自己的时钟和独立的时序控制, 两者之间采用完全的异步工作方式。为此, 在各个外设和主机之间必须要有相应的逻辑部件来解决它们之间的同步与协调、工作速度的匹配和数据格式的转换等问题, 这类逻辑部件统称为 I/O 模块 (有些教材也称为 I/O 接口)。从功能上来说, 各种设备的 I/O 控制器或适配器都是一种 I/O 模块。大型机中的 I/O 模块就是担负大量复杂的外设控制任务的通道或 I/O 处理器。

总线 (bus) 是传输信息的介质, 用于在部件之间传输信息, CPU、主存和 I/O 模块通过总线互连, 在 CPU 和 I/O 模块中都内含相应的存储部件, 即缓存器。

图 1.1 是一个典型的多总线计算机系统的硬件结构示意图。从图 1.1 可以看出, CPU 中包含控制器、算术逻辑部件 ALU、寄存器堆 (register file, 也称通用寄存器组或寄存器文件)、总线接口部件等, CPU、主存储器 and I/O 模块之间通过总线交换信息, 例如, 处理器总线用来传输与 CPU 交换的信息, 存储器总线用来传输与主存储器交换的信息, I/O 总线用来传输与设备控制器交换的信息, 不同总线之间通过I/O 桥接器 (I/O bridge) 相连。CPU 通过处理器总线、I/O 桥接器等与主存储器和 I/O 模块交换信息; 主存储器通过存储器总线、I/O 桥接器与 CPU 和 I/O 模块交换信息; I/O 设备通过各自的设备控制器或适配器连到 I/O 总线上, 例如, 可以把鼠标和键盘连接到 USB 控制器的插口上, 显示器连接到显示适配器的插口上。在一个 I/O 总线上也可以设置多个 I/O 扩展槽, 以连接更多的外设。

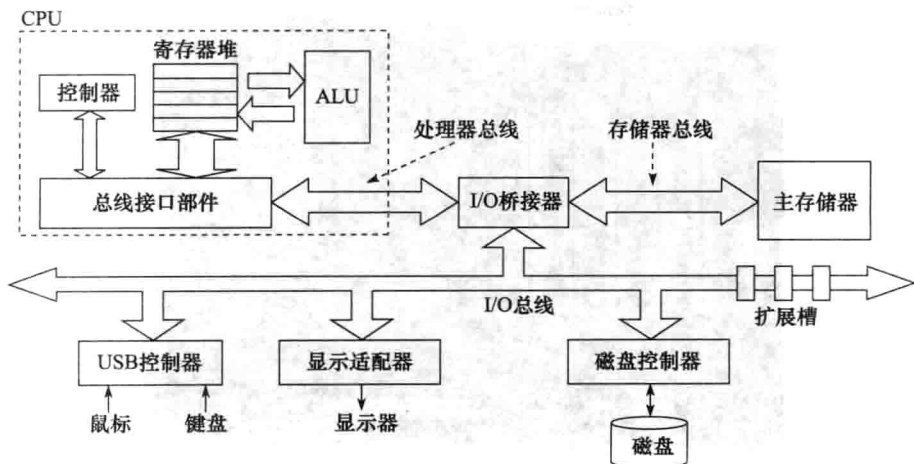


图 1.1 一个典型计算机系统的硬件组成

小贴士

计算机的硬件可以分成主机和外设两部分，主机中的主要功能模块是 CPU、主存和各个 I/O 模块。因为早期计算机的主要功能部件由一条单总线相连，这条总线被称为系统总线，所以，发展为多总线后，就把连接主机中主要功能模块的各类总线统称为系统总线。因此，多总线计算机中的处理器总线、存储器总线和 I/O 总线都属于系统总线。不过，Intel 架构中将连接 CPU 和北桥的处理器总线特指为系统总线，也称为前端总线 FSB (Front Side Bus)。

从外部来看，普通台式个人计算机 (PC) 是用各种电缆将显示器、键盘、鼠标和机箱等连接而成的一个装置。打开一台普通台式机的机箱后，看到的是如图 1.2 所示的一组电路板、芯片和连线，有主板、电源、风扇和硬盘驱动器等。

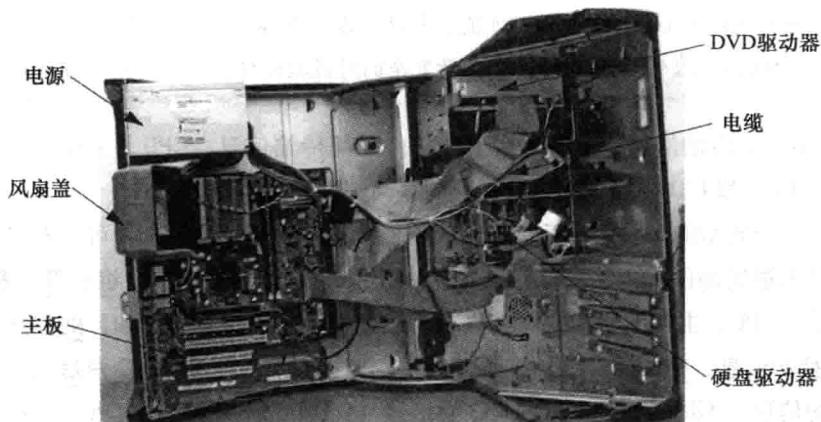


图 1.2 台式个人计算机机箱内的部件

图 1.3 所示的是个人计算机主板，其中有一个处理器芯片插座，用于插入相应的 CPU 芯片，通过 PCI 总线插槽可连接相应外设，内存条也可插入插槽进行扩充和更换。

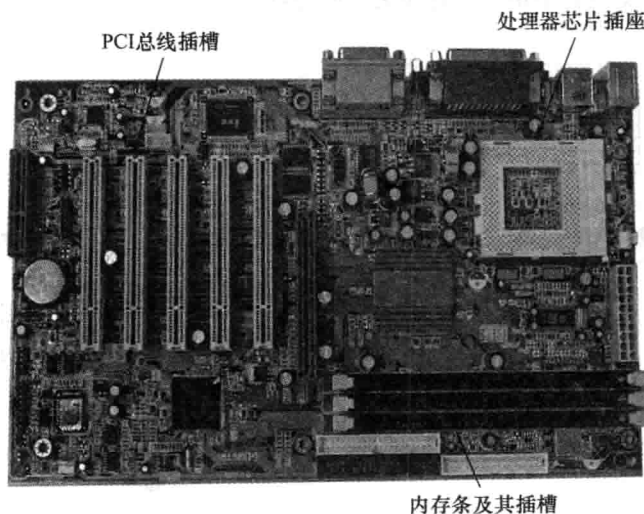


图 1.3 台式个人计算机主板

图 1.4 是对计算机硬件进行解剖的示意图，显示了一台个人计算机的硬件结构分解过程。计算机主机由多个电路板用总线连接而成，每个电路板上又焊接了多个集成电路芯片，每个芯片中有十几个电路模块，每个模块中有上千万个单元，每个单元中有几个门电路，每个门电路实现基本的逻辑运算，因为计算机中所有信息都采用二进制编码表示，二进制的 1 和 0 对应逻辑值“真”和“假”，因此，可以方便地通过逻辑运算电路来实现算术运算。

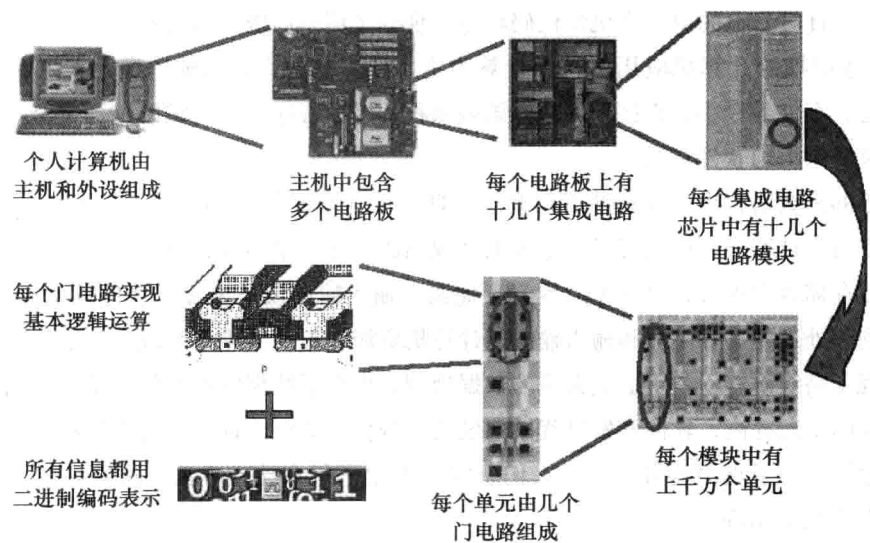


图 1.4 个人计算机的硬件结构解剖

图 1.5 是奔腾 4 处理器芯片的内部结构示意图。左边是芯片的显微照片，右边是功能模块。奔腾 4 处理器芯片中除了整数运算数据通路外，还集成了浮点运算数据通路和多媒体处理数据通路，并且 L1 cache（包括指令 cache 和数据 cache）和 L2 cache 也都制作在 CPU 芯片内。此外，还具有支持高级流水线和超线程的部件，以及用于 I/O 访问和存储器访问的接口部件。

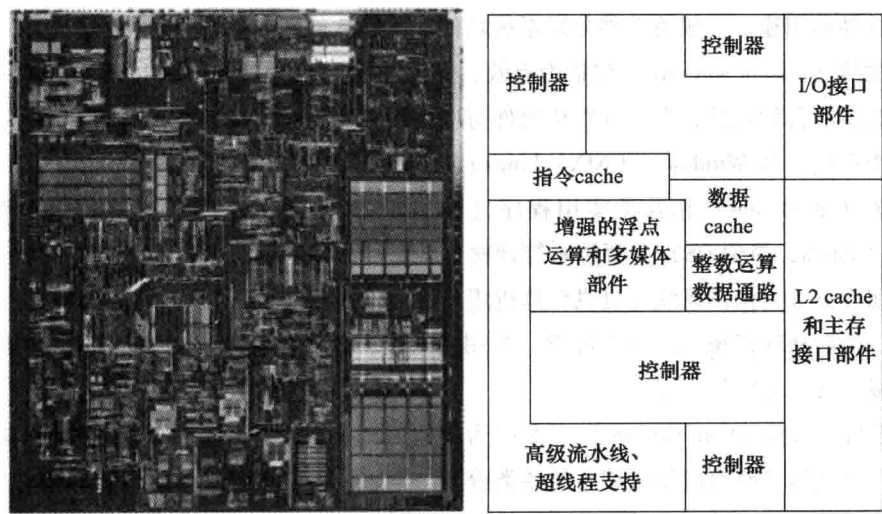


图 1.5 奔腾 4 处理器芯片的内部组成

1.2.3 计算机软件

计算机的工作由存储在其内部的程序控制,这是冯·诺依曼结构计算机“存储程序”工作方式的重要特征,因此程序或者软件质量的好坏将大大影响计算机性能的发挥。

软件的发展受计算机硬件和计算机应用的推动和制约,其发展过程大致分三个阶段。

从第一台计算机上的第一个机器代码程序出现到实用的高级语言出现为第一阶段(1946~1956年)。这时期的计算机应用以科学计算为主,计算量较大,但输入输出量不大;机器以CPU为中心,存储器较小;直接采用机器语言编程,因而程序设计与编制工作复杂、烦琐、易出错。这时尚未出现软件一词。

从实用的高级语言出现到软件工程概念出现以前这段时间为第二阶段(1956~1968年)。这时期除了科学计算外,还出现了大量数据处理问题,计算量不大,但输入输出量较大。机器结构转向以存储器为中心,出现了大容量存储器,输入输出设备增加,软件概念也开始出现。为了充分利用处理器、存储器和输入输出等计算机资源,出现了操作系统;为了提高编程工作效率,出现了高级语言;为了适应大量的数据处理,出现了数据库及其管理系统。随着软件规模和复杂性的不断提高,软件开发过程中问题越来越多,甚至出现了人们难以控制的局面,即所谓软件危机。为了克服这种危机,人们研究和采用了很多技术方法,这就导致了“软件工程”概念和方法的出现。

软件工程出现以后至今一直处于第三阶段。对于一些复杂的大型软件,采用基于个人和简单团队分工的传统开发方式进行开发不仅效率低、可靠性差,且很难完成,必须采用工程方法才能实现。为此,从20世纪60年代末开始,软件工程技术得到了迅速的发展,出现了“计算机辅助软件设计”、“软件自动化”等技术方法和实验系统。目前,人们除了研究改进软件开发技术外,还着重研究具有智能化、自动化、集成化、并行化以及自然化特征的软件新技术。

根据软件的用途,一般将软件分成系统软件和应用软件两大类。

系统软件(system software)包括为有效、安全地使用和管理计算机以及为开发和运行应用软件而提供的各种软件,介于计算机硬件与应用程序之间,它与具体应用关系不大。系统软件包括操作系统(如Windows、UNIX、Linux)、语言处理系统(如Visual Studio、GCC)、数据库管理系统(如Oracle)和各类实用程序(如磁盘碎片整理程序、备份程序)。操作系统(Operating System,简称OS)主要用来管理整个计算机系统的资源,包括对它们进行调度、管理、监视和服务等,操作系统还提供计算机用户和硬件之间的人机交互界面,并提供对应用软件的支持。语言处理系统主要用于提供一个用高级语言编程的环境,包括源程序编辑、翻译、调试、链接、装入运行等功能。

应用软件(application software)指专门为数据处理、科学计算、事务管理、多媒体处理、工程设计以及过程控制等应用所编写的各类程序。例如,人们平时经常使用的电子邮件收发软件、多媒体播放软件、游戏软件、炒股软件、文字处理软件、电子表格软件、演示文稿制作软件等都是应用软件。

1.3 程序开发与执行过程

程序的开发和执行涉及计算机系统的各个不同层面，因而计算机系统层次结构的思想体现在程序开发和执行过程的各个环节中。下面以简单的 hello 程序为例，简要介绍程序的开发与执行过程，以便加深对计算机系统层次结构概念的认识。

编写程序并让其在计算机上执行是为了解决最终用户的应用问题，因而，程序有时被称为用户程序（user program）或应用程序（application program）。

1.3.1 从源程序到可执行程序

以下是 hello.c 的 C 语言源程序代码。

```
1 #include <stdio.h>
2
3 int main()
4 {
5     printf("hello, world\n");
6 }
```

为了让计算机能执行上述应用程序，应用程序员应按照以下步骤进行处理。

① 通过程序编辑软件得到 hello.c 文件。hello.c 在计算机中以 ASCII 字符方式存放，如图 1.6 所示，图中给出了每个字符对应的 ASCII 码的十进制值，例如，第一个字节的值是 35，代表字符‘#’；第二个字节的值是 105，代表字符‘i’，最后一个字节的值为 125，代表字符‘}’。通常把用 ASCII 码字符或汉字字符表示的文件称为文本文件（text file），源程序文件都是文本文件，是可显示和可读的。

#	i	n	c	l	u	d	e	<sp>	<	s	t	d	i	o	.
35	105	110	99	108	117	100	101	32	60	115	116	100	105	111	46
h	>	\n	\n	i	n	t	<sp>	m	a	i	n	(	)	\n	{
104	62	10	10	105	110	116	32	109	97	105	110	40	41	10	123
\n	<sp>	<sp>	<sp>	<sp>	p	r	i	n	t	f	(	"	h	e	l
10	32	32	32	32	112	114	105	110	116	102	40	34	104	101	108
l	o	,	<sp>	w	o	r	l	d	\n	"	)	;	\n	}	
108	111	44	32	119	111	114	108	100	92	110	34	41	59	10	125

图 1.6 hello.c 源程序文件的表示

② 将 hello.c 进行预处理、编译、汇编和链接，最终生成可执行目标文件。例如，在 UNIX 系统中，可用 GCC 编译驱动程序进行处理，命令如下：

```
unix> gcc -o hello hello.c
```

上述命令中，最前面的 unix > 为 shell 命令行解释器 的命令行提示符，gcc 为 GCC 编译驱动程序名，-o 表示后面为输出文件名，hello.c 为要处理的源程序。从 hello.c 到可执行目标文件 hello 的转换过程如图 1.7 所示。

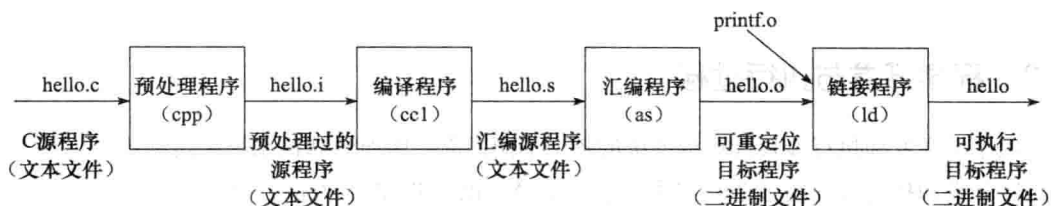


图 1.7 hello.c 源程序文件到可执行目标文件的转换过程

- **预处理阶段**：预处理程序（cpp）对源程序中以字符#开头的命令进行处理，例如，将#include命令后面的.h文件内容嵌入到源程序文件中。预处理程序的输出结果还是一个源程序文件，以.i为扩展名。
- **编译阶段**：编译程序（cc1）对预处理后的源程序进行编译，生成一个汇编语言源程序文件，以.s为扩展名，例如，hello.s是一个汇编语言程序文件。因为汇编语言与具体的机器结构有关，所以，对同一台机器来说，不管什么高级语言，编译转换后的输出结果使用的都是同一种汇编语言。
- **汇编阶段**：汇编程序（as）对汇编语言源程序进行汇编，生成一个可重定位目标文件（relocatable object file），以.o为扩展名，例如，hello.o是一个可重定位目标文件。它是一种二进制文件（binary file），因为其中的代码已经是机器指令，数据以及其他信息也都是用二进制表示的，所以它是不可读的，也即打开显示出来的是乱码。
- **链接阶段**：链接程序（ld）将多个可重定位目标文件和标准库函数合并成为一个可执行目标文件（executable object file），可执行目标文件可简称为可执行文件。本例中，链接程序将hello.o和标准库函数printf所在的可重定位目标模块printf.o进行合并，生成可执行文件hello。

最终生成的可执行文件被保存在磁盘上，可以通过某种方式启动一个磁盘上的可执行文件运行。

1.3.2 可执行文件的启动和执行

对于一个存放在磁盘上的可执行文件，可以在操作系统提供的用户操作环境中，采用双击对应图标或在命令行中输入可执行文件名等多种方式来启动执行。在UNIX系统中，可以通过shell命令行解释器来执行一个可执行文件。例如，对于上述可执行文件hello，通过shell命令行解释器启动执行的结果如下：

```

unix> ./hello
hello, world
unix>
  
```

shell命令行解释器会显示提示符unix>，告知用户它准备接收用户的输入，此时，用户可以在提示符后面输入需要执行的命令名，它可以是一个可执行文件在磁盘上的路径名，例如，上述“./hello”就是可执行文件hello的路径名，其中“./”表示当前目录。在命令后用户需按下[Enter]键表示结束。图1.8显示了在计算机中执行的整个过程。

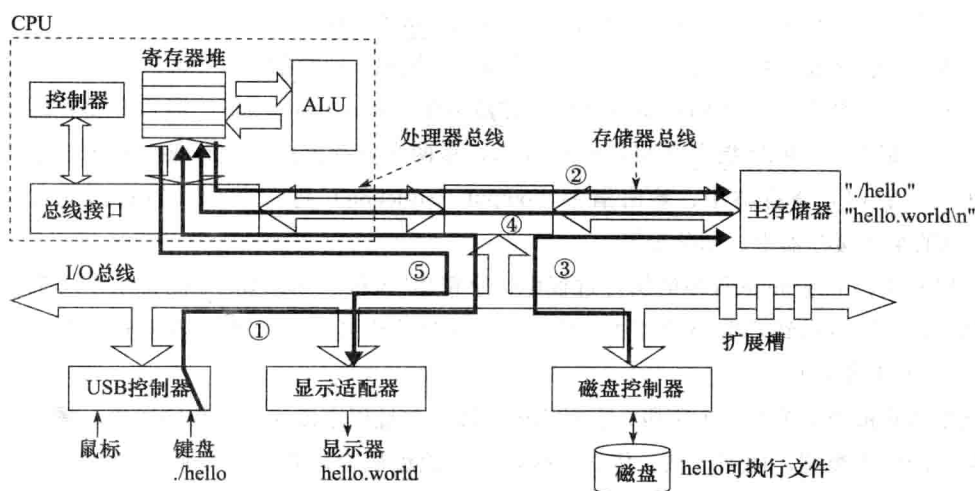


图 1.8 启动和执行 hello 程序的整个过程

如图 1.8 所示，shell 程序会将用户从键盘输入的每个字符逐一读入 CPU 寄存器中（对应线①），然后再保存到主存储器中，在主存的缓冲区形成字符串"./hello"（对应线②）。等到接收到 [Enter] 按键时，shell 将调出操作系统内核中相应的服务例程，由内核来加载磁盘上的可执行文件 hello 到存储器（对应线③）。内核加载完可执行文件中的代码及其所要处理的数据（这里是字符串"hello, world\n"）后，将 hello 第一条指令的地址送到程序计数器（Program Counter，简称 PC）中，CPU 永远都是将 PC 的内容作为将要执行的指令的地址，因此，处理器随后开始执行 hello 程序，它将加载到主存的字符串"hello, world\n"中的每一个字符从主存取到 CPU 的寄存器中（对应线④），然后将 CPU 寄存器中的字符送到显示器上显示出来（对应线⑤）。

从上述过程可以看出，一个用户程序被启动执行，必须依靠操作系统的支持，包括外壳程序和内核服务例程。例如，shell 命令行解释器是操作系统外壳程序，它为用户提供了一个启动程序执行的环境，对用户从键盘输入的命令进行解释，并调出操作系统内核来加载用户程序（用户输入命令对应的程序）。显然，用来加载用户程序并使其从第一条指令开始执行的操作系统内核服务例程也是必不可少的。此外，在上述过程中，涉及键盘、磁盘和显示器等外部设备的操作，这些底层硬件是不能由用户程序直接访问的，此时，也需要依靠操作系统内核服务例程的支持，例如，用户程序需要调用内核的 read 系统调用服务例程读取磁盘文件，或调用内核的 write 系统调用服务例程把字符串写到显示器中等。

从图 1.8 可以看出，程序的执行过程就是数据在 CPU、主存储器和 I/O 模块之间流动的过程，所有数据的流动都是通过总线、I/O 桥接器等进行的。数据在总线上传输之前，需要先缓存在存储部件中，因此，除了主存储器本身是存储部件以外，在 CPU、I/O 桥接器、设备控制器中也有存放数据的缓冲存储部件，例如，CPU 中的寄存器堆、设备控制器中的数据缓冲寄存器等。

1.3.3 程序中每条指令的执行

每个可执行目标文件中都包含机器代码段，可执行文件的执行实际上是对应的机器代码段

执行的过程。机器代码段由一条一条机器指令构成。指令 (instruction) 是用 0 和 1 表示的一串 0/1 序列, 用来指示 CPU 完成一个特定的原子操作, 例如, 取数指令 (load instruction) 从存储单元中取出一个数据存放到 CPU 寄存器中, 存数指令 (store instruction) 将 CPU 寄存器的内容写入一个存储单元, ALU 指令 (ALU instruction) 将两个寄存器内容进行某种算术或逻辑运算后再送入一个 CPU 寄存器中; 输出指令 (output instruction) 将一个 CPU 寄存器的内容送到 I/O 模块的某个缓存器中, 如此等等。

可以看出, 上述 hello 程序的执行过程中, 字符串 "hello, world\n" 中的每个字符都要通过取数指令从存储器送到 CPU 寄存器中, 然后, 再通过输出指令从 CPU 寄存器送到显示适配器 (也称显示控制器) 中。

指令通常被划分为若干个字段, 有操作码字段、地址码字段和立即数字段等。操作码字段指出指令的操作类型, 如加、减、传送、跳转等; 地址码字段指出指令所处理的操作数的地址, 如寄存器编号、内存单元地址等; 立即数字段指出具体的一个操作数或偏移地址等。

图 1.9 给出了实现两个相邻数组元素交换功能的不同层次语言的描述。在高级语言源程序中, 可直观地用三个赋值语句实现; 在经编译后生成的汇编语言源程序中, 可用 4 个汇编指令表示, 其中, 两条是取数指令 lw (load word), 另两条是存数指令 sw (store word); 在经汇编后生成的机器语言程序中, 对应的机器指令是特定格式的二进制代码, 例如, 第一条 lw 指令对应的机器代码为 "1000 1100 0100 1111 0000 0000 0000 0000", 这是一条 MIPS 体系结构中的指令, 其中, 高 6 位 "100011" 为操作码, 随后 5 位 "00010" 为寄存器编号 2, 再后面 5 位 "01111" 为另一个寄存器编号, 最后 16 位表示立即数 0。CPU 能够通过逻辑电路直接执行这种二进制表示的机器指令。指令执行时通过控制器对指令操作码进行译码, 解释成控制信号 (control signal) 控制数据通路执行, 例如, 控制信号 $ALUop = add$ 可以控制 ALU 进行加法操作, $RegWr = 1$ 可以控制将结果写入寄存器。

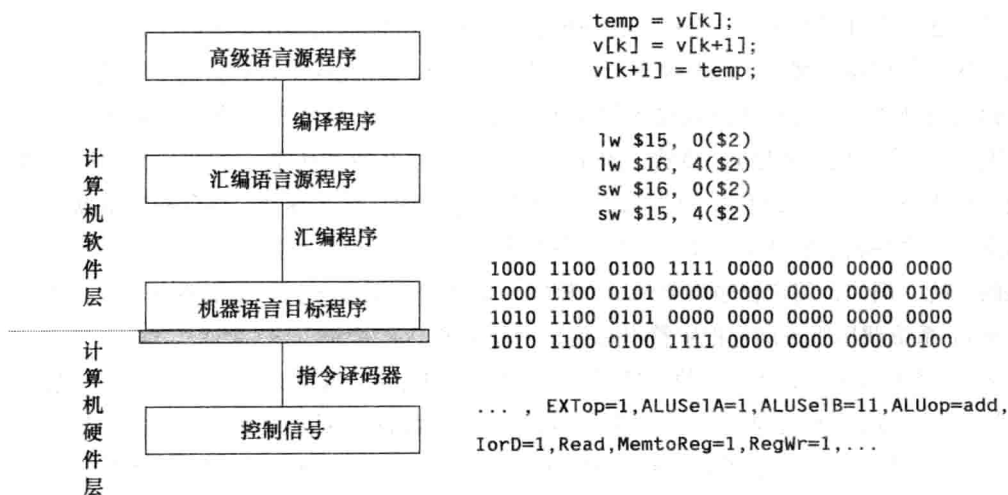


图 1.9 不同层次语言之间的等价转换

每条指令的执行过程包括: 从存储器取指令并计算下一条指令的地址、对指令进行译码、取操作数、对操作数进行运算、送运算结果到存储器或寄存器保存。每次从存储器取指令都是

将 PC 的值作为指令的地址，因此，计算出的下条要执行指令的地址被送到 PC，当前指令执行完后，根据 PC 的值到存储器去取下条指令，从而能够周而复始地执行程序中的每条指令。指令的执行由时钟信号（clock signal）进行定时，一条指令的执行可能需要一个或多个时钟的时间。

1.4 计算机系统的层次结构

传统计算机系统采用分层方式构建，也即计算机系统是一个层次结构系统，通过向上层用户提供一个抽象的简洁接口而将较低层次的实现细节隐藏起来。计算机解决应用问题的过程就是不同抽象层进行转换的过程。

1.4.1 计算机系统抽象层的转换

图 1.10 是计算机系统抽象层次转换示意图，描述了从最终用户希望计算机完成的应用（问题）到电子工程师使用器件完成基本电路设计的整个转换过程。

希望计算机完成或解决的任何一个应用（问题）最开始形成时是用自然语言描述的，但是，计算机硬件只能理解机器语言，而要将一个自然语言描述的应用问题转换为机器语言程序，需要经过多个抽象层的转换。

首先，将应用问题转化为算法（algorithm）描述，使得应用问题的求解变成流程化的清晰步骤，并能确保步骤是有限的。任何一个问题可能有多个求解算法，需要进行算法分析以确定哪种算法在时间和空间上能够得到优化。

其次，将算法转换为用编程语言描述的程序（program），这个转换通常是手工进行的，也就是说，需要程序员进行程序设计。编程语言（programming language）与自然语言不同，它有严格的执行顺序，不存在二义性，能够唯一地确定计算机执行指令的顺序。编程语言可以分成各类不同抽象层的、适用于不同领域的、采用不同描述结构的，等等。目前大约有上千种编程语言。从抽象层次上来分，可以分成高级语言和低级语言两类。高级语言（high-level language）和底层计算机结构关联不大，是机器无关语言，大部分编程语言都是高级语言；低级语言（low-level language）则和运行程序的计算机的底层结构密切相关，通常称为机器级语言（machine-level language），机器语言和汇编语言都是机器级语言。机器语言（machine language）就是用二进制进行编码的机器指令（instruction），每条机器指令都是一个 0/1 序列，因此，机器语言程序的可读性很差，也不易记忆，给程序员的编写和阅读带来极大的困难。因此，人们引入了一种机器语言的符号表示语言，通过用简短的英文符号和二进制代码建立对应关系，以方便程序员编写和阅读机器语言程序，这种语言被称为汇编语言（assembly language）。因为高



图 1.10 计算机系统抽象层及其转换

级语言的可读性比低级语言可读性好得多，所以，绝大部分程序员使用高级语言编写程序。

然后，将高级语言程序转换成计算机能够理解的机器语言程序。因为这个转换过程是计算机自动完成的，因而需要有能够执行自动转换的程序，我们把进行这种转换的软件统称为程序设计语言处理系统。通常，程序员借助程序设计语言处理系统来开发软件。任何一个语言处理系统中，都包含一个翻译程序（translator），它能把一种编程语言表示的程序转换为等价的另一种编程语言程序。被翻译的语言和程序分别称为源语言和源程序，翻译生成的语言和程序分别称为目标语言和目标程序。翻译程序有以下三类。

① 汇编程序（assembler）：也称汇编器，用来将汇编语言源程序翻译成机器语言目标程序。

② 解释程序（interpreter）：也称解释器，用来将源程序中的语句按其执行顺序逐条翻译成机器指令并立即执行。

③ 编译程序（compiler）：也称编译器，用来将高级语言源程序翻译成汇编语言或机器语言目标程序。

当然，所有的语言处理系统都必须在操作系统提供的计算机环境中运行，操作系统是对计算机底层结构和计算机硬件的一种抽象，这种抽象构成了一台可以让程序员使用的虚拟机（virtual machine）。

从应用问题到机器语言程序的每次转换所涉及的概念都是属于软件的范畴，而机器语言程序所运行的计算机硬件和软件之间需要有一个“桥梁”，这个在软件和硬件之间的界面就是指令集体系结构（Instruction Set Architecture, ISA），简称体系结构或系统结构（architecture），它是软件和硬件之间接口的一个完整定义。ISA 定义了一台计算机可以执行的所有指令的集合，每条指令规定了计算机执行什么操作，以及所处理的操作数存放的地址空间和操作数类型。ISA 规定的内容包括：数据类型及格式，指令系统格式，寻址方式和可访问地址空间大小，程序可访问的寄存器个数、位数和编号，控制寄存器的定义，I/O 空间的编址方式，中断结构，机器工作状态的定义和切换，输入/输出结构和数据传送方式，存储保护方式等。因此，可以看出，指令集体系结构是指软件能感知到的部分，也称软件可见部分。

机器语言程序就是一个 ISA 规定的指令的序列，因此，计算机硬件执行机器语言程序的过程就是让其执行一条一条指令的过程。ISA 是对指令系统的一种规定或结构规范，具体实现的组织（organization）称为微体系结构（microarchitecture），简称微架构。ISA 和微体系结构是两个不同层面上的概念，微体系结构是软件不可感知的部分。例如，加法器采用串行进位方式还是并行进位方式实现属于微体系结构。相同的 ISA 可能具有不同的微体系结构，例如，对于 Intel x86 这种 ISA，很多处理器的组织方式不同，也即具有不同的微体系结构，但因为它们具有相同的 ISA，所以一种处理器上运行的程序，在另一种处理器上也能运行。

微体系结构最终是由逻辑电路（logic circuit）实现的，当然，微架构中的一个功能部件可以用不同的逻辑来实现，用不同的逻辑实现方式得到的性能和成本是有差异的。

最后，每个基本的逻辑电路都是按照特定的器件技术（device technology）实现的，例如，CMOS 电路中使用的器件和 NMOS 电路中使用的器件不同。

1.4.2 计算机系统的不同用户

按照在计算机上完成任务的不同,可以把使用计算机的用户分成以下四类:最终用户、系统管理员、应用程序员和系统程序员。

使用应用程序完成特定任务的计算机用户称为最终用户 (end user)。大多数计算机使用者都属于最终用户。例如,使用炒股软件的股民、玩计算机游戏的人、进行会计电算化处理的财会人员等。

系统管理员 (system administrator) 是指利用操作系统等软件提供的功能对系统进行配置、管理和维护,以建立高效合理的系统环境供计算机用户使用的操作人员。其职责主要包括:安装、配置和维护系统的硬件和软件,建立和管理用户账户,升级软件,备份和恢复业务系统和数据等。

应用程序员 (application programmer) 是指使用高级编程语言编制应用软件的程序员;而系统程序员 (system programmer) 则是指设计和开发系统软件的程序员,如开发操作系统、编译器、数据库管理系统等系统软件的程序员。

很多情况下,同一个人可能既是最终用户,又是系统管理员,同时还是应用程序员或系统程序员。例如,对于一个计算机专业的学生来说,有时需要使用计算机玩游戏或网购物品,此时为最终用户的角色;有时需要整理计算机磁盘中的碎片、升级系统或备份数据,此时是系统管理员的角色;有时需要完成老师布置的开发一个应用程序的作业,此时是应用程序员的角色;有时可能还需要完成老师布置的开发操作系统或编译程序等软件的作业,此时是系统程序员的角色。

计算机系统采用层次化体系结构,不同用户工作在不同的系统结构层,所看到的计算机的概念性结构和功能特性是不同的。

1. 最终用户

早期的计算机非常昂贵,只能由少数专业化人员使用。随着20世纪80年代初个人计算机的迅速普及以及20世纪90年代初多媒体计算机的广泛应用,特别是互联网技术的发展,计算机已经成为人们日常生活中的重要工具。人们利用计算机播放电影,玩游戏,炒股票,发邮件,查信息,聊天打电话等,计算机的应用无处不在。因而,许多普通人都成为了计算机的最终用户。

计算机最终用户使用键盘和鼠标等外设与计算机交互,通过操作系统提供的用户界面启动执行应用程序或系统命令,从而完成用户任务。因此,最终用户能够感知到的只是系统提供的简单人机交互界面和安装在计算机中的相关应用程序。

2. 系统管理员

相对于普通的计算机最终用户,系统管理员作为管理和维护计算机系统的专业人员,对计算机系统的了解要深入得多。系统管理员必须能够安装、配置和维护系统的硬件和软件,能建立和管理用户账户,需要时能升级硬件和软件,备份和恢复业务系统和数据等。也就是说,系统管理员应该非常熟悉操作系统提供的有关系统配置和管理方面的功能,很多普通用户解决不了的问题,系统管理员必须能够解决。

因此,系统管理员能感知到的是系统中部分硬件层面、系统管理层面以及相关的实用程序和人机交互界面。

3. 应用程序员

应用程序员大多使用高级程序设计语言编写程序。所谓高级程序设计语言 (high level programming language) 是指面向算法设计的较接近于日常所用的英语书面语言的程序设计语言,例如 BASIC、C/C++、Fortran、Java 等。

应用程序员所看到的计算机系统除了计算机硬件、操作系统提供的应用编程接口 (API)、人机交互界面和实用程序外,还包括相应的程序语言处理系统。

在语言处理系统中,除了翻译程序外,通常还包括编辑程序、链接程序、装入程序以及将这些程序和工具集成在一起所构成的集成开发环境 (Integrated Development Environment, 简称 IDE) 等。此外,语言处理系统中还包括可供应用程序调用的各类函数库。

4. 系统程序员

系统程序员开发操作系统、编译器和实用程序等系统软件时,需要熟悉计算机底层的相关硬件和系统结构,甚至可能需要直接与计算机硬件和指令系统打交道。比如,直接对各种控制寄存器、用户可见寄存器、I/O 控制器等硬件进行控制和编程。因此,系统程序员必须熟悉指令系统、机器结构和相关的机器功能特性,有时还要直接用汇编语言等低级语言编写程序代码。

在计算机技术中,一个存在的事物或概念从某个角度看似乎不存在,也即,对实际存在的事务或概念感觉不到,则称为透明。通常,在一个计算机系统中,系统程序员所看到的底层机器级的概念性结构和功能特性对高级语言程序员 (通常就是应用程序员) 来说是透明的,也即看不见或感觉不到的。因为对应用程序员来说,他们直接用高级语言编程,不需要了解有关汇编语言的编程问题,也不用了解机器语言中规定的指令格式、寻址方式、数据类型和格式等指令系统方面的问题。

一个计算机系统可以认为是由各种硬件和各类软件采用层次化方式构建的分层系统,不同计算机用户工作所在的系统结构层如图 1.11 所示。

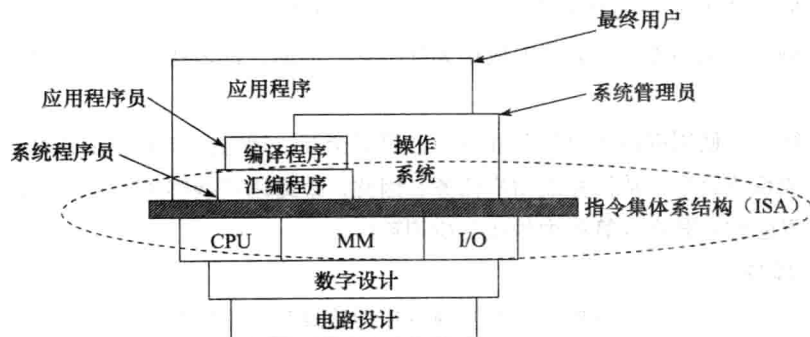


图 1.11 计算机系统的层次化结构

从图 1.11 中可看出,ISA 处于硬件和软件的交界面上,硬件所有的功能都由 ISA 集中体现,软件通过 ISA 在计算机上执行。所以,ISA 是整个计算机系统核心部分。

ISA 层下面是硬件部分，上面是软件部分。硬件部分包括 CPU、主存和输入/输出等主要功能部件，这些功能部件通过数字逻辑电路设计实现。软件部分包括低层的系统软件和高层的应用软件，汇编程序、编译程序和操作系统等系统软件直接在 ISA 上实现，系统程序员所看到的机器的属性是属于 ISA 层面的内容，所看到的机器是配置了指令系统的机器，称为机器语言机器，工作在该层次的程序员称为机器语言程序员；系统管理员工作在操作系统层，所看到的是配置了操作系统的虚拟机器，称为操作系统虚拟机；汇编语言程序员工作在提供汇编程序的虚拟机器级，所看到的机器称为汇编语言虚拟机；应用程序员大多工作在提供编译器或解释器等翻译程序的语言处理系统层，因此，应用程序员大多用高级语言编写程序，因而也称为高级语言程序员，所看到虚拟机器称为高级语言虚拟机；最终用户则工作在最上面的应用程序层。

1.5 计算机系统性能评价

一个完整的计算机系统由硬件和软件构成，硬件性能的好坏对整个计算机系统的性能起着至关重要的作用。硬件的性能检测和评价比较困难，因为硬件的性能只能通过运行软件才能反映出来，而在相同硬件上运行不同类型的软件，或者同样的软件用不同的数据集进行测试，所测到的性能都可能不同。因此，必须有一套综合的测试和评价硬件性能的方法。

1.5.1 计算机性能的定义

吞吐率 (throughput) 和 响应时间 (response time) 是考量一个计算机系统性能的两个基本指标。吞吐率表示在单位时间内所完成的工作量，类似的概念是 带宽 (bandwidth)，它表示单位时间内所传输的信息量。响应时间是指从作业提交开始到作业完成所用的时间，类似的概念是 执行时间 (execution time) 和 等待时间 (latency)，它们都是用来表示一个任务所用时间的度量值。

不同应用场合下，计算机用户所关心的性能是不同的。例如，在多媒体应用场合，用户希望音/视频的播放要流畅，即单位时间内传输的数据量要大，因而关心的是系统吞吐率是否高；而在银行、证券等事务处理应用场合，用户希望业务处理速度快，不需长时间等待，因而更关心响应时间是否短；还有些应用场合（如 ATM、文件服务、Web 服务等），用户则同时关心吞吐率和响应时间。

1.5.2 计算机性能的测试

如果不考虑应用背景而直接比较计算机性能，则大都用程序的执行时间来衡量。也即，从执行时间来考虑，完成同样工作量所需时间最短的那台计算机性能是最好的。

操作系统在对处理器进行调度时，一段时间内往往会让多个程序（更准确地说是进程）轮流使用处理器，因此在某个用户程序执行过程中，可能同时还会有其他用户程序和操作系统程序在执行，所以，通常情况下，一个程序的执行时间除了程序包含的指令在 CPU 上执行所用的时间外，还包括磁盘访问时间、存储器访问时间、输入输出操作所需时间以及操作系统运行这个程序所用的额外开销等。也即，用户感觉到的某个程序的执行时间并不是其真正的执行

时间。通常把用户感觉到的执行时间分成以下两部分：CPU 时间和其他时间。CPU 时间指 CPU 用于本程序执行的时间，它又包括以下两部分：① 用户 CPU 时间，指真正用于运行用户程序代码的时间；② 系统 CPU 时间，指为了执行用户程序而需要 CPU 运行操作系统程序的时间。其他时间指等待 I/O 操作完成的时间或 CPU 用于执行其他用户程序的时间。

计算机系统的性能评价主要考虑的是 CPU 性能。系统性能和 CPU 性能不等价，两者有一些区别。系统性能是指系统的响应时间，它与 CPU 外的其他部分也有关系；而CPU 性能是指用户 CPU 时间，它只包含 CPU 运行用户程序代码的时间。

在对 CPU 时间进行计算时需要用到以下几个重要的概念和指标。

① 时钟周期：计算机执行一条指令的过程被分成若干步骤和相应的动作来完成，每一步动作都要有相应的控制信号进行控制，这些控制信号何时发出、作用时间多长，都要有相应的定时信号进行同步。因此，CPU 必须能够产生同步的时钟定时信号，也就是 CPU 的主脉冲信号，其宽度称为时钟周期（clock cycle, tick, clock tick, clock）。

② 时钟频率：CPU 的主频就是 CPU 中的主脉冲信号的时钟频率（clock rate），是 CPU 时钟周期的倒数。

③ CPI：CPI（Cycles Per Instruction）表示执行指令所需的时钟周期数。由于不同指令的功能不同，所需的时钟周期数也不同，因此，对于一条特定指令而言，其 CPI 指执行该条指令所需的时钟周期数，此时 CPI 是一个确定的值；对于一个程序或一台机器来说，其 CPI 指该程序或该机器指令集中的所有指令执行所需的平均时钟周期数，此时，CPI 是一个平均值。

已知上述参数或指标，可以通过以下公式来计算用户程序的 CPU 执行时间，即用户 CPU 时间。

用户 CPU 时间 = 程序总时钟周期数 ÷ 时钟频率 = 程序总时钟周期数 × 时钟周期

上述公式中，程序总时钟周期数可由程序总指令条数和相应的 CPI 求得。

如果已知程序总指令条数和综合 CPI，则可用如下公式计算程序总时钟周期数。

程序总时钟周期数 = 程序总指令条数 × CPI

如果已知程序中共有 n 种不同类型的指令，第 i 种指令的条数和 CPI 分别为 C_i 和 CPI_i ，则

$$\text{程序总时钟周期数} = \sum_{i=1}^n (CPI_i \times C_i)$$

程序的综合 CPI 也可由以下公式求得，其中， F_i 表示第 i 种指令在程序中所占的比例。

$$CPI = \sum_{i=1}^n (CPI_i \times F_i) = \text{程序总时钟周期数} \div \text{程序总指令条数}$$

因此，若已知程序综合 CPI 和总指令条数，则可用下列公式计算用户 CPU 时间。

用户 CPU 时间 = CPI × 程序总指令条数 × 时钟周期

有了用户 CPU 时间，就可以评判两台计算机性能的好坏。计算机的性能可以看成是用户 CPU 时间的倒数，因此，两台计算机性能之比就是用户 CPU 时间之比的倒数。若计算机 M1 和 M2 的性能之比为 n ，则说明“计算机 M1 的速度是计算机 M2 的速度的 n 倍”，也就是说“在计算机 M2 上执行程序的时间是在计算机 M1 上执行时间的 n 倍”。

用户 CPU 时间度量公式中的时钟周期、指令条数、CPI 三个因素是相互制约的。例如，更改指令集可以减少程序总指令条数，但是，同时可能引起 CPU 结构的调整，从而可能会增加

时钟周期的宽度（即降低时钟频率）。对于解决同一个问题的不同程序，即使是在同一台计算机上，指令条数最少的程序也不一定执行得最快。有关时钟周期、指令条数和 CPI 的相互制约关系，在学完后面有关章节后，会有更深刻的认识和理解。

例 1.1 假设某个频繁使用的程序 P 在机器 M1 上运行需要 10 秒钟，M1 的时钟频率为 2GHz。设计人员想开发一台与 M1 具有相同 ISA 的新机器 M2。采用新技术可使 M2 的时钟频率增加，但同时也会使 CPI 增加。假定程序 P 在 M2 上的时钟周期数是在 M1 上的 1.5 倍，则 M2 的时钟频率至少达到多少才能使程序 P 在 M2 上的运行时间缩短为 6 秒钟？

解 程序 P 在机器 M1 上的时钟周期数为：用户 CPU 时间 $\times$ 时钟频率 $= 10\text{s} \times 2\text{GHz} = 20\text{G}$ 。因此，程序 P 在机器 M2 上的时钟周期数为 $1.5 \times 20\text{G} = 30\text{G}$ 。要使程序 P 在 M2 上运行时间缩短到 6s，则 M2 的时钟频率至少应为：程序总时钟周期数 $\div$ 用户 CPU 时间 $= 30\text{G}/6\text{s} = 5\text{GHz}$ 。

由此可见，M2 的时钟频率是 M1 的 2.5 倍，但 M2 的速度却只是 M1 的 1.67 倍。 ■

上述例子说明，由于时钟频率的提高可能会对 CPU 结构带来影响，从而使其他性能指标降低，因此，虽然时钟频率提高会加快 CPU 执行程序的速度，但不能保证执行速度有相同倍数的提高。

例 1.2 假设计算机 M 的指令集中包含 A、B、C 三类指令，其 CPI 分别为 1、2、4。某个程序 P 在 M 上被编译成两个不同的目标代码序列 P1 和 P2，P1 所含 A、B、C 三类指令的条数分别为 8、2、2，P2 所含 A、B、C 三类指令的条数分别为 2、5、3。请问：哪个代码序列总指令条数少？哪个执行速度快？它们的 CPI 分别是多少？

解 P1 和 P2 的总指令条数分别为 12 和 10，所以，P2 的总指令条数少。

P1 的总时钟周期数为 $8 \times 1 + 2 \times 2 + 2 \times 4 = 20$ 。

P2 的总时钟周期数为 $2 \times 1 + 5 \times 2 + 3 \times 4 = 24$ 。

因为两个指令序列在同一台机器上运行，所以时钟周期一样，故总时钟周期数少的代码序列所用时间短、执行速度快。显然，P1 比 P2 快。

从上述结果来看，总指令条数少的代码序列执行时间并不更短。

$\text{CPI} = \text{程序总时钟周期数} \div \text{程序总指令条数}$ ，因此，P1 的 CPI 为 $20/12 = 1.67$ ；P2 的 CPI 为 $24/10 = 2.4$ 。 ■

上述例子说明，指令条数少并不代表执行时间短，同样，时钟频率高也不说明执行速度快。在评价计算机性能时，仅考虑单个因素是不全面的，必须三个因素同时考虑。1.5.3 节介绍的性能指标 MIPS 曾被普遍使用，它就没有考虑所有三个因素，所以用它来评价性能有时会得到不准确的结论。

1.5.3 用指令执行速度进行性能评估

最早用来衡量计算机性能的指标是每秒钟完成单个运算（如加法运算）指令的条数。当时大多数指令的执行时间是相同的，并且加法指令能反映乘、除等运算性能，其他指令的时间大体与加法指令相当，故加法指令的速度有一定的代表性。指令速度所用的计量单位为 MIPS (Million Instructions Per Second)，其含义是平均每秒钟执行多少百万条指令。

早期还有一种类似于 MIPS 的性能估计方式,就是指令平均执行时间,也称等效指令速度法或Gibson 混合法。随着计算机体系结构的发展,不同指令所需的执行时间差别越来越大,人们就根据等效指令速度法通过统计各类指令在程序中所占比例进行折算。设某类指令 i 在程序中所占比例为 w_i , 执行时间为 t_i , 则等效指令的执行时间为: $T = w_1 \times t_1 + w_2 \times t_2 + \cdots + w_n \times t_n$ (n 为指令种类数)。若指令执行时间用时钟周期数来衡量的话, 则上式计算的结果就是 CPI。对指令平均执行时间求倒数能够得到 MIPS 值。

选取一组指令组合,使得得到的平均 CPI 最小,由此得到的 MIPS 就是峰值 MIPS (peak MIPS)。有些制造商经常将峰值 MIPS 直接当作 MIPS,而实际上的性能要比标称的性能差。

相对 MIPS (relative MIPS) 是根据某个公认的参考机型来定义的相应 MIPS 值,其值的含义是被测机型相对于参考机型 MIPS 的倍数。

MIPS 反映了机器执行定点指令的速度,但是,用 MIPS 来对不同的机器进行性能比较有时是不准确或不客观的。因为不同机器的指令集不同,而且指令的功能也不同,也许在机器 M1 上某一条指令的功能,在机器 M2 上要用多条指令来完成,因此,同样的指令条数所完成的功能可能完全不同;另外,不同机器的 CPI 和时钟周期也不同,因而同一条指令在不同机器上所用的时间也不同。下面的例子可以说明这点。

例 1.3 假定某程序 P 编译后生成的目标代码由 A、B、C、D 四类指令组成,它们在程序中所占的比例分别为 43%、21%、12%、24%,已知它们的 CPI 分别为 1、2、2、2。现重新对程序 P 进行编译优化,生成的新目标代码中 A 类指令条数减少了 50%,其他类指令的条数没有变。请回答下列问题。

① 编译优化前后程序的 CPI 各是多少?

② 假定程序在一台主频为 50MHz 的计算机上运行,则优化前后的 MIPS 各是多少?

解 优化后 A 类指令的条数减少了 50%,因而各类指令所占比例分别计算如下。

A 类指令: $21.5 / (21.5 + 21 + 12 + 24) = 27\%$

B 类指令: $21 / (21.5 + 21 + 12 + 24) = 27\%$

C 类指令: $12 / (21.5 + 21 + 12 + 24) = 15\%$

D 类指令: $24 / (21.5 + 21 + 12 + 24) = 31\%$

① 优化前后程序的 CPI 分别计算如下。

优化前: $43\% \times 1 + 21\% \times 2 + 12\% \times 2 + 24\% \times 2 = 1.57$

优化后: $27\% \times 1 + 27\% \times 2 + 15\% \times 2 + 31\% \times 2 = 1.73$

② 优化前后程序的 MIPS 分别计算如下。

优化前: $50\text{M} / 1.57 = 31.8\text{MIPS}$

优化后: $50\text{M} / 1.73 = 28.9\text{MIPS}$

从 MIPS 数来看,优化后程序执行速度反而变慢了。

这显然是错误的,因为优化后只减少了 A 类指令条数而其他指令数没变,所以程序执行时间一定减少了。从这个例子可以看出,用 MIPS 数来进行性能估计是不可靠的。

与定点指令运行速度 MIPS 相对应的用来表示浮点操作速度的指标是 MFLOPS (Million

FLOating-point operations Per Second)。它表示每秒所执行的浮点运算有多少百万 (10^6) 次, 它是基于所完成的操作次数而不是指令数来衡量的。类似的浮点操作速度还有 GFLOPS (10^9)、TFLOPS (10^{12})、PFLOPS (10^{15}) 和 EFLOPS (10^{18}) 等。

1.5.4 用基准程序进行性能评估

基准程序 (benchmark) 是进行计算机性能评测的一种重要工具。基准程序是专门用来进行性能评价的一组程序, 能够很好地反映机器在运行实际负载时的性能, 可以通过在不同机器上运行相同的基准程序来比较在不同机器上的运行时间, 从而评测其性能。基准程序最好是用用户经常使用的一些实际程序, 或是某个应用领域的一些典型的简单程序。对于不同的应用场合, 应该选择不同的基准程序。例如, 对用于软件开发的计算机进行评测时, 最好选择包含编译器和文档处理软件的一组基准程序; 而如果是对于 CAD 处理的计算机进行评测, 最好选择一些典型的图形处理小程序作为一组基准程序。

基准程序是一个测试程序集, 由一组程序组成。例如, SPEC 测试程序集是应用最广泛, 也是最全面的性能评测基准程序集。1988 年, 由 Sun、MIPS、HP、Apollo、DEC 五家公司联合提出了 SPEC 标准。它包括一组标准的测试程序、标准输入和测试报告。这些测试程序是一些实际的程序, 包括系统调用、I/O 等。最初提出的基准程序集分成两类: 整数测试程序集 SPECint 和浮点测试程序集 SPECfp。后来分成了按不同性能测试用的基准程序集, 如 CPU 性能测试集 (SPEC CPU2000)、Web 服务器性能测试集 (SPECweb99) 等。

如果基准测试程序集中不同的程序在两台机器上测试得出的结论不同, 则如何给出最终的评价结论呢? 例如, 假定基准测试程序集包含有程序 P1 和 P2, 程序 P1 在机器 M1 和机器 M2 上运行的时间分别是 10 秒和 2 秒, 程序 P2 在机器 M1 和机器 M2 上运行的时间分别是 120 秒和 600 秒。也即, 对于 P1, M2 的速度是 M1 的 5 倍; 而对于 P2, M1 的速度是 M2 的 5 倍。那么, 到底是 M1 快还是 M2 快呢? 可以用所有程序的执行时间之和来比较, 例如, P1 和 P2 在 M1 上的执行时间总和为 130 秒, 而在 M2 上的总时间为 602 秒, 故 M1 比 M2 快。但通常不这样做, 而是采用执行时间的算术平均值或几何平均值来综合评价机器的性能。如果考虑每个程序的使用频度而用加权平均的方式, 结果会更准确。

也可以将执行时间进行归一化来得到被测试的机器相对于参考机器的性能。

执行时间的归一化值 = 参考机器上的执行时间 ÷ 被测机器上的执行时间

例如, SPEC 比值 (SPEC ratio) 是指将测试程序在 Sun SPARCstation 上运行时的执行时间除以该程序在测试机器上的执行时间所得到的比值。比值越大, 机器的性能越好。

使用基准程序进行计算机性能评测也存在一些缺陷, 因为基准程序的性能可能与某一小段的短代码密切相关, 此时, 硬件系统设计人员或编译器开发者可能会针对这些代码片段进行特殊的优化, 使得执行这段代码的速度非常快, 以至于得到不准确的性能评测结果。例如, Intel Pentium 处理器运行 SPECint 时用了公司内部使用的特殊编译器, 使其性能表现得很高, 但用户实际使用的是普通编译器, 达不到所标称的性能。又如, 矩阵乘法程序 SPECmatrix300 有 99% 的时间运行在一行语句上, 有些厂商用特殊编译器优化该语句, 使性能达到 VAX 11/780 的 729.8 倍!

1.6 本书的主要内容和组织结构

本书主要介绍与计算机系统相关的核心基本概念，解释这些概念是如何相互关联并最终影响程序执行的结果和性能的。本书将围绕高级语言程序开发和执行所涉及的、与底层机器级代码的运行相关的内容展开。具体内容包括：程序中处理的数据在机器中的表示和运算，程序中各类控制语句对应的机器级代码的结构，可执行目标代码的链接生成，可执行目标代码中的指令序列在机器上的执行过程，存储访问操作过程，打断程序正常执行的机制，程序中的 I/O 操作功能如何通过请求操作系统内核提供的系统调用服务来完成等。

本书以高级语言程序为出发点来组织内容，按照“自顶向下”的方式，即高级语言程序→汇编语言程序→机器指令序列→控制信号的顺序，展现程序从编程设计、翻译转换、链接，到最终运行的整个过程。在关于程序的运行方面，不仅介绍了用于执行程序中指令的 CPU 的基本组成，还详细描述了在指令执行环节中的存储访问机制和异常控制流实现机制，此外，还全面介绍了程序中 I/O 操作的底层实现机制。

对于存储访问机制和异常控制流这两部分内容，本书在介绍基本原理的基础上，还简要介绍了 IA-32/Linux 平台的具体实现（书中带 * 的章节）。由于基本原理在一个具体平台中的实现往往比较复杂，因而带 * 章节的内容相对较难。若本书用作教材的话，这部分可以选择不作为课堂教学内容。但是，如果后续的操作系统课程实验内容是基于 IA-32/Linux 平台实现的话，建议将这部分内容作为重点讲解。

本书第 2 章和第 3 章分别介绍高级语言程序中的数据和语句所对应的底层机器级表示，展示的是高级语言程序到机器级语言程序的对应转换关系；第 4 章主要介绍如何将不同的程序模块链接起来构成可执行目标文件，展示的是程序的链接环节；第 5 章和第 6 章着重介绍程序的运行环节，包括与程序运行密切相关的硬件部分——CPU 及存储器的组织；第 7 章介绍打断程序正常运行的事件机制——异常控制流；第 8 章主要介绍程序中 I/O 操作的实现机制。

本书各章的主要内容说明如下。

第 1 章 计算机系统概述

主要介绍计算机的发展历程、计算机系统的基本功能和基本组成、程序的开发与执行过程、计算机系统层次结构以及计算机性能评价的基本概念。

第 2 章 数据的机器级表示与处理

主要介绍各类数据在计算机中的表示与运算。计算机中的数据只能用 0 和 1 表示，而且运算部件只有有限位数，因而计算机中的算术运算与现实中的算术运算有所区别。例如，一个整数的平方可能为负数；一个负整数可能比一个正整数大；两个正整数的乘积可能比乘数小；浮点数运算时可能不满足结合律。计算机算术运算的这些特性使得有些程序产生意想不到的结果，甚至造成安全漏洞，许多程序员为此感到困惑和苦恼。本章将从数据的机器级表示及其基本运算电路层面来解释计算机算术运算的本质特性，从而使程序员能够清楚地理解由于计算机算术的局限性而造成的异常程序行为。

第3章 程序的转换及机器级表示

主要介绍高级语言中的过程调用和控制语句（如选择、循环等结构语句）所对应的汇编指令序列，以及各类数据结构（如数组、指针、结构、联合等）元素的访问所对应的汇编指令序列。高级语言程序员使用高度抽象的过程调用、控制语句和数据结构等来实现算法，因而无法了解程序在计算机系统中执行的细节，无法真正理解程序设计中的许多抽象概念，也就很难解释清楚某些程序的行为和执行结果。本章在机器级的汇编指令层面来解释程序的行为，因而能对程序执行结果进行较为清楚的说明。通过本章学习，将会明白诸如以下一些问题：过程调用时按值传递参数和按地址传递参数的差别在哪里？缓冲区溢出的漏洞是如何造成的？为什么递归调用会耗内存？为什么同样的程序在32位架构上和64位架构上执行的结果会不同？指针操作的本质是什么？

第4章 程序的链接

主要介绍如何将多个程序模块链接起来生成一个可执行目标文件。通过介绍与链接相关的可重定位目标文件格式、符号解析、重定位、静态库、共享目标库以及可执行目标文件的加载等内容，使程序员清楚地了解哪些问题是与链接相关的。例如，程序一些意想不到的结果是由于变量在多个模块中的多重定义造成的；链接时可能存在一些无法解析的符号是与指定的输入文件顺序有关的。此外，链接生成的可执行目标文件与程序加载、虚拟地址空间和存储空间映射等重要内容相关，对于理解操作系统中的存储管理方面的内容非常有用。

第5章 程序的执行

前面第2章到第4章介绍了将高级语言源程序进行预处理、编译、汇编和链接形成可执行目标文件的过程，因而本章顺其自然地简要介绍可执行目标文件中的机器代码如何在计算机中执行。因为执行程序的底层硬件结构非常复杂，涉及的内容较多，而本书的篇幅有限，所以无法展开细说。本章主要介绍程序以及指令的执行过程、CPU的基本功能和基本组成、数据通路的基本组成和工作原理、流水线方式下指令的执行过程。

第6章 层次结构存储系统

在第5章介绍的程序执行机制中，有一个重要的概念就是指令以及指令处理的某些操作数存放在存储器中，因而在执行指令过程中，需要通过访问存储器来取指令或读写操作数。通常，程序员以为程序代码和数据按序存放在一个由线性地址构成的主存空间中，实际上计算机中的存储器并不是只有主存，而是主存与其他种类存储部件（如高速缓存、硬盘等）共同构成的一个层次结构存储系统。因此，在后续章节中提到的“存储单元”不一定是指主存单元，“访存过程”也不是仅指访问主存的过程，而是访问整个存储系统的过程。本章将介绍如何构成一个层次结构存储系统以及在层次结构存储系统中的访存过程。层次结构存储系统能获得较好效果的一个很重要的原因是，程序中的存储访问具有局部性特点，因此，本章将详细介绍如何通过改善程序的时间局部性和空间局部性来提高程序执行的性能。

第7章 异常控制流

在程序正常执行过程中，CPU会因为遇到内部异常事件或外部中断事件而打断原来程序的执行，转去执行操作系统提供的针对这些特殊事件的处理程序。这种由于某些特殊情况引起用户程序的正常执行被打断所形成的意外控制流称为异常控制流（Exceptional Control of Flow，

ECF)。显然,计算机系统必须提供一种机制使得自身能够实现异常控制流。本章主要介绍硬件层和操作系统层中涉及的对于内部异常和外部中断的异常控制流实现机制,主要内容包括进程与进程上下文切换、异常的类型、异常的捕获和处理、中断的捕获和处理、系统调用的实现机制等。

第8章 I/O 操作的实现

所有高级语言的运行时系统都提供了执行 I/O 功能的高级机制,例如,C 语言中提供了像 `fread`、`printf` 和 `scanf` 等这样的标准 I/O 库函数,C++ 语言中提供了如 `<<` (输入) 和 `>>` (输出) 这样的重载操作符。从用户在高级语言程序中通过 I/O 函数或 I/O 操作符提出 I/O 请求,到设备响应并完成 I/O 请求,整个过程涉及多个层次的 I/O 软件和 I/O 硬件的协调工作。本章主要介绍与 I/O 操作相关的软件和硬件方面的相关内容,主要包括文件的概念、I/O 系统调用函数、C 标准 I/O 库函数、设备控制器的基本功能和结构、I/O 端口的编址方式、外设与主机之间的 I/O 控制方式以及如何利用“陷阱指令”将用户 I/O 请求转换为 I/O 硬件操作的过程。

不管构建一个计算机系统的各类硬件和软件多么千差万别,计算机系统的构建原理以及在计算机系统上的程序转换和执行机理是相通的,因而,本书仅介绍一种特定计算机系统平台下的相关内容。本书所用的平台为 IA-32/x86-64 + Linux + GCC + C 语言。

1.7 小结

本章主要对计算机系统作了概述性的介绍,指出了本书内容在整个计算机系统的位置,介绍了计算机系统的基本功能和基本组成、计算机系统各个抽象层之间的转换以及程序开发和执行的概要过程,并对计算机系统的性能评价作了简要说明。

计算机在控制器的控制下,能完成数据处理、数据存储和数据传输三个基本功能,因而它由完成相应功能的控制器、运算器、存储器、输入和输出设备组成。在计算机内部,指令和数据都用二进制表示,两者形式上没有任何差别,都是一个 0/1 序列,它们都存放在存储器中,按地址访问。计算机采用“存储程序”方式进行工作。计算机系统采用逐层向上抽象的方式构成,通过向上层用户提供一个抽象的简洁接口而将较低层次的实现细节隐藏起来。在底层系统软件和硬件之间的抽象层就是指令集体系结构 (ISA),或简称体系结构。硬件和软件相辅相成,缺一不可,两者都可用来实现逻辑功能。

计算机完成一个任务的大致过程如下:用某种程序设计语言编制源程序;用语言处理程序将源程序翻译成机器语言目标程序;将目标程序中的指令和数据装入内存,然后从第一条指令开始执行,直到程序所含指令全部执行完。每条指令的执行包括取指令、指令译码、取操作数、运算、送结果等操作。

计算机系统基本性能指标包括响应时间、吞吐率。处理器的基本性能参数包括时钟周期 (或主频)、CPI、MIPS、MFLOPS 等。一般把程序的响应时间划分成 CPU 时间和其他时间,CPU 时间又分成用户 CPU 时间和系统 CPU 时间。因为操作系统对自己所花费的时间进行测量时,不十分准确,所以,对 CPU 性能的测量一般通过测量用户 CPU 时间来进行。

习题

1. 给出以下概念的解释说明。

中央处理器 (CPU)	算术逻辑部件 (ALU)	数据通路	控制器
主存	系统软件	应用软件	高级语言
汇编语言	机器语言	机器级语言	源程序
目标程序	编译程序	解释程序	汇编程序
操作系统	最终用户	系统管理员	应用程序员
系统程序员	指令系统	指令集体系结构 (ISA)	微体系结构
透明	响应时间	吞吐率	用户 CPU 时间
系统 CPU 时间	系统性能	CPU 性能	时钟周期
主频	CPI	基准程序	SPEC 基准程序集
SPEC 比值	MIPS	峰值 MIPS	相对 MIPS
MFLOPS			

2. 简单回答下列问题。

- (1) 冯·诺依曼计算机由哪几部分组成？各部分的功能是什么？采用什么工作方式？
 - (2) 摩尔定律的主要内容是什么？
 - (3) 计算机系统的层次结构如何划分？计算机系统的用户可分哪几类？每类用户工作在哪个层次？
 - (4) 程序的 CPI 与哪些因素有关？
 - (5) 为什么说性能指标 MIPS 不能很好地反映计算机的性能？
3. 假定你的朋友不太懂计算机，请用简单通俗的语言给你的朋友介绍计算机系统是如何工作的。
4. 你对计算机系统的哪些部分最熟悉，哪些部分最不熟悉？最想进一步了解细节的是哪些部分的内容？
5. 若有两个基准测试程序 P1 和 P2 在机器 M1 和 M2 上运行，假定 M1 和 M2 的价格分别是 5000 元和 8000 元，下表给出了 P1 和 P2 在 M1 和 M2 上所花的时间和指令条数。

程序	M1		M2	
	指令条数	执行时间	指令条数	执行时间
P1	200×10^6	10 000ms	150×10^6	5000ms
P2	300×10^3	3ms	420×10^3	6ms

请回答下列问题：

- (1) 对于 P1，哪台机器的速度快？快多少？对于 P2 呢？
- (2) 在 M1 上执行 P1 和 P2 的速度分别是多少 MIPS？在 M2 上的执行速度又各是多少？从执行速度来看，对于 P2，哪台机器的速度快？快多少？

- (3) 假定 M1 和 M2 的时钟频率各是 800MHz 和 1.2GHz, 则在 M1 和 M2 上执行 P1 时的平均时钟周期数 CPI 各是多少?
- (4) 如果某个用户需要大量使用程序 P1, 并且该用户主要关心系统的响应时间而不是吞吐率, 那么, 该用户需要大批购进机器时, 应该选择 M1 还是 M2? 为什么? (提示: 从性价比上考虑。)
- (5) 如果另一个用户也需要购进大批机器, 但该用户使用 P1 和 P2 一样多, 主要关心的也是响应时间, 那么, 应该选择 M1 还是 M2? 为什么?
6. 若机器 M1 和 M2 具有相同的指令集, 其时钟频率分别为 1GHz 和 1.5GHz。在指令集中有五种不同类型的指令 A ~ E。下表给出了在 M1 和 M2 上每类指令的平均时钟周期数 CPI。

机器	A	B	C	D	E
M1	1	2	2	3	4
M2	2	2	4	5	6

请回答下列问题:

- (1) M1 和 M2 的峰值 MIPS 各是多少?
- (2) 假定某程序 P 的指令序列中, 五类指令具有完全相同的指令条数, 则程序 P 在 M1 和 M2 上运行时, 哪台机器更快? 快多少? 在 M1 和 M2 上执行程序 P 时的平均时钟周期数 CPI 各是多少?
7. 假设同一套指令集用不同的方法设计了两种机器 M1 和 M2。机器 M1 的时钟周期为 0.8ns, 机器 M2 的时钟周期为 1.2ns。某个程序 P 在机器 M1 上运行时的 CPI 为 4, 在 M2 上的 CPI 为 2。对于程序 P 来说, 哪台机器的执行速度更快? 快多少?
8. 假设某机器 M 的时钟频率为 4GHz, 用户程序 P 在 M 上的指令条数为 8×10^9 , 其 CPI 为 1.25, 则 P 在 M 上的执行时间是多少? 若在机器 M 上从程序 P 开始启动到执行结束所需的时间是 4 秒, 则 P 占用的 CPU 时间的百分比是多少?
9. 假定某编译器对某段高级语言程序编译生成两种不同的指令序列 S1 和 S2, 在时钟频率为 500MHz 的机器 M 上运行, 目标指令序列中用到的指令类型有 A、B、C 和 D 四类。四类指令在 M 上的 CPI 和两个指令序列所用的各类指令条数如下表所示。

	A	B	C	D
各指令的 CPI	1	2	3	4
S1 的指令条数	5	2	2	1
S2 的指令条数	1	1	1	5

请问: S1 和 S2 各有多少条指令? CPI 各为多少? 所含的时钟周期数各为多少? 执行时间各为多少?

10. 假定机器 M 的时钟频率为 1.2GHz, 某程序 P 在机器 M 上的执行时间为 12 秒。对 P 优化时, 将其所有的乘 4 指令都换成了一条左移两位的指令, 得到优化后的程序 P'。已知在 M 上乘 4 指令的 CPI 为 5, 左移指令的 CPI 为 2, P 的执行时间是 P' 执行时间的 1.2 倍, 则 P 中有多少条乘法指令被替换成了左移指令来执行?